Axona

Architecture

David A. Smith
Axona.net

An Axona Architecture Note v4.59.2 2026-08-05

Written against `@axona/protocol` **v4.59.2**, commit `30664923e844`, deployed on testnet and production.
Bridge `2.105.1` Relay `v0.100.0`

For human readers and AI implementers alike. Section II states what this document does not yet let you build; read it before relying on anything here for a clean-room implementation.

A protocol is the contract between layers that don't trust each other.

What I cannot create, I do not understand.
Richard Feynman · 1988

I What This Network Is For

Axona is a communication network with no owner — no company, no server, no place in the middle where anyone sits who could read, rank, charge, throttle, or forbid. Not because an operator promises restraint, but because by construction there is nowhere for one to be. It runs in production on ordinary phones, browsers, and laptops.

The keystone is one design choice: **authorship is a signature, not an account.** Who is speaking is a key the speaker holds. Where they are and how they connect is a separate fact the network never links to it. The network moves signed bytes between endpoints and does nothing else — so it *cannot act on* what you say. It cannot rank it, suppress it, or reveal who you are, because it was never told and never needed to know.

That is the whole property. Everything below is machinery for keeping it true under churn, partition, and load.

This section is deliberately short and deliberately first. An implementer who knows only the frames will make locally reasonable choices that violate the thesis — adding a return address to make debugging easier, caching an identity to save a round trip. Those choices are the failure mode this section exists to prevent.

What the thesis has already decided

These follow from the property above, not from taste. An implementation that breaks one is not a variant of Axona; it is a different network wearing the same wire format. Each has been re-litigated at least once and settled.

- **The signed envelope discloses *who*, never *where*.** No node id, no region, no address in the envelope — ever. This was proposed twice for good-looking reasons and reverted twice. The routing wrapper is a different object with different rules (§III).
- **Transport identity does not persist.** A stable node id across restarts is a correlator, and a correlator plus an IP is a physical location. Restart must yield a different node id. Author identity is the opposite: it persists, because provenance is the point.
- **Region is an optimization, never a wall.** The geographic byte places traffic; it never gates admission or capability. A sparse region must roll over to its neighbours rather than refuse service. A node that cannot be served locally is still served.
- **No content-based anything in the kernel.** No ranking, no content filtering, no priority by payload. The kernel does not model

what a message says and must not acquire the ability. Moderation is a subscriber-side choice over signed objects, never a network-side one.

- **Retraction is the author's, not the network's.** A creator can kill their own message and the network will carry the tombstone. Nobody else can, and there is no operator who could be compelled to.
- **No accounts, no directory, no operator.** Bootstrap uses bridges, and a bridge is a convenience that the protocol must survive losing. If any mechanism here only works while a bridge is up, that is a defect, not a design.

What it costs

A network that cannot be made to take sides cannot be made to take the right side either. The same property that lets a researcher under sanctions collaborate as an equal lets a disinformation campaign route around every attempt to slow it. It gives AI agents from different vendors somewhere to meet as peers, and it gives a misaligned agent the same place on the same terms.

This document does not resolve that, and no clever framing should be expected to. The judgement is that a tool no one can control is worth building for the same reason humanity kept fire and built hearths and fire brigades rather than giving it back — and that the answer to such a tool is not a central authority who controls it, since that only rebuilds the intermediary the design set out to remove.

Where this note is silent and you must choose, choose the option that keeps the network unable to act on content, unable to link authorship to location, and able to keep running when any particular participant vanishes. That ordering has settled every genuine ambiguity so far.

II How to Read This Document

This note is two documents sharing one text. Read as prose, it explains what Axona is and why each mechanism exists — every rule in this system was paid for by a specific failure, and the failures are named. Read as a specification, it contains the architectural model and the current wire contract: identifier derivations (§VI), the complete wire vocabulary with payload schemas (§IX, §X), the root-management decision table (§XI), the timing model (§XIII), and the invariants any implementation must uphold (§XIV). Section XX is addressed directly to an implementer — human or AI — and orders

the work. It will not yet carry you to a peer that interoperates byte-for-byte with the deployed network; §III says exactly where it stops.

Three conventions. *Normative* material (schemas, constants, invariants, algorithms) appears in tables and numbered rules; prose around it is *narrative* and explains intent. Feynman's epigraph is the acceptance test: a reader who cannot re-derive the mechanism from this note has found a bug in the note, and should report it as such. And every constant is given with its value *and* its reason — a constant whose reason you don't know is one you will mis-tune.

Companion notes: *Root Management (RootClaim state machine)* for the full election deep-dive; the *Kernel Refactor Analysis* for how the code reached its current shape; the *Soak-Test Framework* overview for how the live network is characterized. The invariants used to live in a separate `INVARIANTS.md` — in two separate files, in fact, under two numbering schemes. They are now §XIV of this document and nowhere else.

III What This Document Does Not Claim

A specification that overstates itself is worse than one that stops early, because the reader cannot tell which parts to trust. On 2026-08-04 a reconstruction audit read the previous revision against the v4.59.2 source and found five claims the code does not support. They are corrected in the sections below. This table is where they live now, so that no reader has to discover them by building the wrong thing.

Subject	**What the code does**	**Do not claim**
Geography	The region byte is chosen by the node and is *not* covered by the admission signature; only the 256-bit hash suffix is bound to the public key (§VI).	That a node's location is cryptographically proven. It is an area code, not an attestation.
Author replay	The root verifies the envelope signature and a signed-timestamp freshness window, then deduplicates by `msgId` within that topic's retained state.	Monotonic per-author sequence enforcement at live ingress. There is no author high-water mark.
Routing privacy	The signed envelope carries no node ID. The routing wrapper that moves it does: every `route_msg` carries `originId`.	That a publish has no return address. Envelope privacy and routing-metadata privacy are different properties.
Durability	A cohort gate exists and is counted, but local delivery to a self-subscribed publisher confirms the pending publish before that gate completes — a known deviation, fenced by `test/fence_q2_end_to_end.mjs`.	That seeing your own message proves it replicated.
Ordering	A root stamps a dense per-topic sequence. Partitioned roots stamp independently, and delivery overwrites the publisher's own sequence with the root's.	A network-wide or partition-spanning total order, or that the publisher's sequence survives to the application.

Beyond these, three things are simply not published yet: machine-readable schemas for every frame, golden test vectors for canonicalization and the authentication transcript, and the bridge signalling contract. Until those exist, an independent implementation can be built from this note but cannot be *verified* against the network without reading the source. §XX states the order in which that gap closes.

IV The System in One Page

Axona is a peer-to-peer message substrate: publish/subscribe over a distributed hash table, with no servers in the data path. Every participant — a browser tab, a phone, a headless Node process — is a *peer* holding a keypair. Peers connect to each other directly over WebRTC data channels (browsers) or WebSockets (servers), discover each other through the DHT, and exchange *routed messages*: each message is addressed to a 264-bit identifier and forwarded, hop by hop, to the live peer whose own identifier is closest to the target by XOR distance.

Everything else is built from that one primitive:

- A **topic** is a 264-bit id derived from its name, owner, and write policy. Publishing routes the message toward the topic id; the peer where routing terminates is the topic's **root** — emergent, never elected. The root timestamps (*stamps*) each message, giving the topic a total order, caches recent history, and fans deliveries out to subscribers.
- A **subscription** is a periodically renewed routed message toward the same topic id. The renewal is simultaneously the keepalive, the failure detector, the gap-recovery request, and the self-heal: if anything about the path or the root has changed, the renewal finds the new truth and re-attaches.
- **Durability** is replication among the peers XOR-closest to the topic (the *cohort*), plus explicit hand-off when a root departs gracefully.
- **Identity** is cryptographic and self-authenticating: a message is believed because it verifies against the author key it carries, never because of who relayed it. There is no certificate authority, no account database, and no server that can revoke a publisher.

A network has any number of bridge servers, each for *signaling only*: a bridge introduces new peers so they can open direct connections. Bridges federate by uplinking to each other as ordinary peers, advertise themselves on a public directory topic, and clients rank the directory and fail over (§VIII). A bridge is bootstrap infrastructure, not a data-path relay; two peers with established mesh connections continue exchanging messages if every bridge is down, and new connections can even be signaled across the existing mesh.

Proven live 2026-07-12: a fresh authenticated data channel formed between two peers with the bridge process dead, signaling relayed over the mesh (19/19, bulk 100%).

The design bets on one asymmetry, everywhere: *wrong claims that self-correct are cheaper than coordination that prevents them.* Roots are claimed optimistically and reconciled by evidence; publishes are retried idempotently rather than acknowledged; caches converge by

anti-entropy rather than consensus. The protocol has no leader, no quorum, and no distributed lock anywhere.

The bet has a boundary, and the boundary was paid for in production: the optimism applies to *claims* — statements that evidence can cheaply revoke — never to *standing state*. A wrong claim costs its holder one demotion; wrong standing state costs whoever must maintain or evict it, forever if no one does. So claims are optimistic, but state obeys two laws stated as invariants below: it is bounded by demand, never by churn history (I-10), and it may be planted on another node only by a principal alive to maintain it (the principal-liveness rule, §X).

The 4.24.0 release let a departing node plant replica state on its cohort as a durability fallback — locally reasonable, and it collapsed the testnet backbone twice in one night. The post-mortem is Kernel-Refactor-Analysis v0.2 §1.

V The Stack

Layer	Repo / package	Role
Kernel	`@axona/protocol`	identity, routing, transports, pub/sub
Bridge	`axona-bridge`	WebSocket signaling + TURN minting; embedded peer
Relay	`axona-relay`	headless supernode (hosting, metrics, ops CLI)
Reference app	`axona-chat`	reference browser application (axona.chat)
Simulator	`dht-sim`	1K–50K-node simulation over the vendored kernel

The kernel is the protocol; everything else consumes it. Its pub/sub internals were consolidated in the 4.19–4.21 refactor into single-concern modules behind one façade — the module map doubles as the concept map of this document:

Module (`src/pubsub/`)	Lines	Owns
`AxonaManager.js`	∼500	façade: state, public API, routing egress
`wireHandlers.js`	∼760	routed handlers + axon-tree mechanics
`repairPlane.js`	∼550	the tick scheduler, retries, replication, departure
`rootElection.js`	∼290	beacons, root hints, self-verification, liveness
`rootClaim.js`	∼240	the `isRoot` state machine (§XI)
`constants.js`	∼210	every tunable + wire types (§XIII)
`topicStore.js`	∼150	cache, tombstones, exactly-once app delivery
`envelope.js / post.js / kill.js`	—	signing, canonical form, ids

Outside pub/sub, the kernel provides `src/dht/AxonaPeer.js` (the peer object: lifecycle, the local node, routing entry points, the public API of §XVII), `src/transport/` (web, node, sim transports; handshake; mesh management), `src/identity/`, and `src/utils/hexid.js` (the keyspace profile: 264-bit production ids, reducible for simulation).

VI Identity and Addressing

Everything in Axona lives in one keyspace: **264-bit identifiers**, written on the wire as 66 lowercase hex characters, handled internally as `BigInt`. An id is always `regionByte` ‖ `256-bit body`. The high byte is an S2 geographic cell (a coarse region code); the remaining 256 bits are cryptographic material. Distance between ids is XOR, compared

as unsigned integers; the region byte's position makes same-region ids numerically close, so proximity in the keyspace loosely tracks proximity on the planet.

The S2 cube yields 192 cells, but roughly half are open ocean or near-empty land where no peers live. Minting an id there would be a mistake: a topic anchored in empty water has no local population, so the nodes nearest its region byte — a boundary sliver of the closest real region — would absorb *every* topic anchored anywhere in that ocean, a hotspot. So every id is minted through `canonicalRegion()`, which *folds* each ocean or sparse cell onto its nearest **major** (populated) region — 84 of them. A node in mid-Pacific and a topic anchored there both resolve to Hawaii's region and spread across its full node population. The fold is deterministic, carried in the id itself (no coordinator), and wire-compatible: every populated region keeps its exact byte. Regions are labelled with neutral in-range *animal* names (`eagle`, `chinkara`, `penguin`) rather than country names — a country name on a border-spanning cell is needlessly political, and the label is presentation only (the id carries the numeric code, never the name).

Sim A/B (2,100 nodes, 2,000 ocean-anchored topics): the fold cuts peak per-node load 2.7× and busiest-1% share 1.9×.

Node identity (where routing can reach you)

`createNodeIdentity({lat, lng})` generates an Ed25519 keypair and derives `nodeId = regionByte(lat,lng)` ‖ `SHA-256(pubkey)`. The node key authenticates *transport*: channel establishment, handshakes, routing — never content. Node identity is typically ephemeral; a peer that restarts with a fresh key is simply a new node.

Author identity (who said it)

`createAuthorIdentity()` generates a separate, location-free Ed25519 keypair; the `authorId` is the public key. Author keys sign *content* and are usually persistent (the browser app stores them in IndexedDB, non-extractable). The separation is deliberate and load-bearing: the network learns *where* a node is from its node id but only *who* authored a message from its envelope — a signed envelope discloses the author, never the author's location, and no protocol change may add a node-id or region to the envelope.

This rule has been defended twice against convenience patches. Location display, where an application wants it, is app-level and opt-in.

Topics (structured addresses with policy folded in)

A topic is addressed by a *descriptor* {`region`, `owner`, `name`, `write`}: `region` a region name or code; `owner` an authorId or null; `name` a UTF-8 string; `write` either `'open'` (anyone publishes) or `'owner'` (only the owner's key). The id is:

```
topicId = regionByte
        || SHA-256( canonical({ owner, name, write }) )
```

with the region byte resolved by rule: an explicit `region` wins; otherwise the *publisher's own node region* is used; otherwise the derivation throws (`TOPIC_REGION_REQUIRED`). The region is **never** derived from the author key — the author has no location, and hashing the authorId into a region would dump every one of an author's topics into one arbitrary cell, a hotspot. (An earlier design did exactly that and was removed in 3.1.0.) Whichever rule fires, the resulting byte passes through `canonicalRegion()`, so a topic can never be anchored at an unpopulated cell.

`write` defaults on whether an owner is named, and the default is the safe one: no owner → the topic is necessarily `'open'` (any passed `write` is ignored); owner named → `'owner'` unless `'open'` is passed explicitly (an owner-namespaced inbox anyone may post to). Forgetting `write` on an owned feed must not leave it world-writable (3.2.0). Because the write policy is folded into the hash, a topic's policy is not mutable state — a root recomputes the id from the signed descriptor and enforces `write:'owner'` statelessly.

Messages (content-addressed, author-signed)

Every published message travels as an **envelope**:

```
{ msgId:  hex64,           // content address, see below
  seq:    int,             // per-publisher monotonic
  ts:     ms-epoch,        // publisher clock
  topic:  { region, owner, name, write },   // the signed descriptor
  message: <any JSON value>,
  signature?:   'ed25519:' + hex128,
  signerPubkey?: hex64 }    // the author key (absent = anonymous)
```

- `msgId = SHA-256(canonical({publisher, message}))` where `publisher` is `signerPubkey` or null — a stable content address of (author, message), independent of time, topic, or signature. Identical re-publishes collapse wherever both copies meet retained state for the *same topic* — dedup is topic-local and retention-bounded, not network-wide, and the same body under two topics is two messages. A publisher wanting distinct ids adds a nonce to `message`.
- The signature covers the domain-tagged core `canonical({d, seq, ts, topic, message})` with `d` the envelope domain tag — domain separation means an envelope signature can never be replayed as a kill signature or a channel-binding value; signing the resolved descriptor binds the message to the exact topic *and policy*.
- `canonical()` is a total, JSON-valid canonical encoding with recursively sorted keys, shared by every signing and hashing site in the system. It is *not* RFC 8785: it inherits JavaScript `JSON.stringify` value semantics, including dropped `undefined` object members and coerced non-finite numbers. An independent implementation must

reproduce those semantics, not JCS. Golden vectors are not yet published (§III).

- Freshness: receivers enforce a bound on `ts` skew at live ingress, then deduplicate by `msgId` against that topic's retained cache and tombstones. There is *no* per-author sequence high-water mark — the anti-replay property is the signed timestamp window plus topic-local id dedup, and it is bounded by retention. Replayed history is exempt from the ts bound but capped by a future-tolerance rule (§X).

A **kill** (retraction) is its own signed object naming `(topicId, msgId)`, domain-tagged, and valid only when signed by *the same author key* that signed the target message — creator-only retraction, enforced at every root, with a provisional path when the kill races ahead of its target (§X).

VII Routing

Each peer maintains a **synaptome**: a bounded table (target ∼50 entries) of live, authenticated, directly-connected neighbours, mixing keyspace-near peers (completeness around one's own id) with long-range contacts (reach). Entries are first-party only — gossip feeds a *candidate pool*, and a candidate becomes a synapse only after a verified direct connection, which is the eclipse defense: no one can write into another peer's routing table.

Two search primitives share the table:

Greedy routing (`routeMessage`) forward to the neighbour strictly closest to the target; the peer where no neighbour improves on self is the *terminal*. One-pass, no backtracking, hop-budgeted. Fast and usually right; on a sparse or divergent mesh it can strand in a local minimum — every convergence mechanism in §XI exists to correct exactly this.

Iterative lookup (`findKClosest` / `lookup`) the classic α-parallel iterative search, querying successively closer peers for their neighbours. Slower (multiple round trips) but global: it escapes local minima and crosses regions. The kernel *never* blocks a send on it — lookups warm hints in the background (§XI).

Two standing rules. **The bridge is never a data hop**: routing skips the bridge id even when it is numerically closest — signaling infrastructure must not become a root, a relay, or a convergence anchor. **Region is a placement hint, not a wall**: the region byte clusters same-region ids, but a lookup crosses regions freely; treating the prefix

as a routing wall caused a cross-region split-brain (fixed 4.17.1) and is now an explicit anti-pattern.

Departed peers are marked dead immediately on channel loss (eviction, graceful-leave notify) and skipped by routing; reachability to a *newcomer* lives in its neighbours' tables and is healed by inbound traffic — which is why cold publishers re-send by design (§XIII) rather than waiting to be found.

One property of greedy routing surprises every instrument pointed at it: **a routed message need not touch the wire at all**. The origin resolves its next hop and, finding none closer than itself, delivers locally at hop zero — so when a publisher happens to be the topic-closest node, its publish is consumed in-process and no PUB frame exists anywhere. This is correct, it is how a solo node roots its own topics, and it happens to roughly one publish in N on an N-node network, at random, by topology. A test that asserts "a publish produces traffic" is therefore wrong one time in N; a packet capture at the publisher shows the *replication* frames that follow and none of the publish itself. Measure delivery at a subscriber, never presence on a wire.

VIII Transports and the Bridge

The kernel is transport-agnostic behind one interface; three implementations ship. `Transport.web()` — WebRTC data channels between peers, WebSocket to the bridge for signaling; the browser production path. `Transport.node()` — WebSocket server-to-server; relays and bridges. `Transport.sim()` — in-process, for the simulator and tests.

Authentication: `axona/5`

Every channel is mutually authenticated before any Axona frame flows. The handshake proves possession of the node key via a signed, domain-tagged challenge bound to the channel: for WebRTC, the channel-binding value folds both sides' *DTLS certificate fingerprints* (a MITM terminating DTLS changes the fingerprints and the signature fails); for WebSockets, the binding uses the connection's nonces. A peer's claimed nodeId must carry `SHA-256(pubkey)` as its low 256 bits for the proven key. The *region byte is deliberately not covered*: a node picks its own area code, and admission neither verifies nor cares where it claims to be. Identity is verified; *location is asserted*, and the difference matters to any threat model that reasons about regional concentration.

Version discipline

Two version axes ride the hello frames. `WIRE_VERSION` (currently `4.0`) partitions hermetically on its major — wire-3.x and wire-4.x reject each other at both the bridge gate and the peer-peer handshake, because 4.x changed convergence and replay semantics. `KERNEL_VERSION` (currently `4.59.2`) is informative and surfaced at every runtime surface (healthz, welcome, UI) — an observability rule, not a gate; wire-compatible kernels interoperate freely, and deploys are staggered on exactly that property.

The bridge, and life without it

The bridge accepts WebSocket connections, runs the version gate and the authenticated hello, mints short-lived TURN credentials, and *introduces* peers: a joining peer receives a set of live peers to open WebRTC connections to, and the bridge ferries the SDP/ICE signaling for those first connections. The bridge embeds an ordinary kernel peer (it hosts keyspace, participates in routing for signaling purposes) but is excluded from data-path roles by every selection rule.

Join flow: connect WSS → version gate → authenticated hello → welcome (peer list, TURN) → open direct channels → synaptome admission per connection → the peer is routable. `connect()` wraps the whole sequence in one call. After bootstrap the bridge is optional: *mesh-relayed signaling* routes SDP/ICE across existing data channels, so new direct connections form bridge-free, and existing connections never depended on it.

The bridge can also say so out loud. A meshed client is a slot the bridge no longer needs, so the bridge closes it with a *graduation* code (4200). A client holding at least the graduation floor of authenticated mesh peers keeps its mesh and stops reconnecting; a watchdog re-dials the bridge only if the mesh later thins. A client that was not actually meshed reconnects immediately — which makes the whole scheme self-correcting, so the bridge can graduate optimistically and let the client's own state decide. Before 4.35.0 the client read the graduation close as a failure and tore its mesh down to zero peers on the way out — the bridge politely dismissing a student and the student burning down the school.

Bridges self-advertise on the public topic `axona:bridge-directory` (each bridge `host()`s and republishes it); clients collect the directory at launch, rank, and fail over. Multiple bridges federate by uplinking to each other as ordinary peers, so their peer populations form one mesh.

Judging clients fairly

A bridge judges client liveness only over time it was actually listening: an event-loop stall on the bridge re-arms client grace and suspends idle sweeps until the taint ages out (invariant I-5). A client must never be dropped for the server's own pause.

IX The Wire Protocol

All frames are JSON. Above the transport layer there are three frame families: the handshake/hello family (§VIII), direct notifications (`direct_*`, point-to-point over an existing channel), and *routed messages* — the workhorse. A routed message carries `{type, payload, targetId, hops, originId}` and is forwarded greedily toward `targetId`. The handler registered for `type` runs *at every hop*, not only at the terminal: each receiver dispatches locally — with `meta.isTerminal` telling it which it is — and only then decides whether to forward. Waypoint behaviour is a consequence of that, not a special case.

`originId` travels with every routed frame. The signed envelope carries no node id, but the wrapper moving it does; see §III.

Pub/sub message types. This list is **complete and normative** — all seventeen routed types the v4.59.2 kernel emits or accepts. The per-frame *payload schemas* are not: they are described in prose in §X and are not yet published as machine-readable schemas with required/optional fields, caps, and receiver-validation verdicts. An implementer can name every frame from this table and still be unable to construct several of them. That gap is §III's first item and closes with the conformance vectors.

A missing type is worse than an undocumented one: it makes an implementation look conformant while silently dropping a verb. `pubsub:pullresp` was absent from this table until the 2026-08-04 audit read it out of `constants.js`.

Type	Routed toward	Meaning
`pubsub:sub`	topic (or via)	subscribe / renew; carries since, hw, lw
`pubsub:unsub`	topic (or via)	explicit unsubscribe
`pubsub:pub`	topic (or via)	publish; unstamped envelope
`pubsub:deliver`	subscriber	stamped messages (and del markers)
`pubsub:adopt`	child	delegate: become child relay, take batch
`pubsub:pullup`	holder	"you are ahead of me — replay up"
`pubsub:replayup`	parent	stamped cache delta, upward
`pubsub:handoff`	heir	departing root pushes history
`pubsub:handoffack`	leaver	heir confirms receipt (quenches retry/fallback)
`pubsub:replicate`	cohort member	*live* root pushes cache+tombstones (or empty keepalive)
`pubsub:rootbeacon`	neighbours	soft-state root advertisement
`pubsub:kill`	topic (or via)	signed retraction
`pubsub:pull`	topic (or via)	one-shot read request
`pubsub:pullresp`	requester	the read answer, correlated by `corrId`
`pubsub:metricson`	topic (or via)	demand-driven metrics lease
`pubsub:touch`	topic	reserved no-op (wire compat)
`pubsub:unpub`	—	reserved string, no handler

Via waypoints. Topic-addressed payloads may carry `via: [hex,…]` (capped at 8), an ordered waypoint list: the message routes toward `via[0]` first. At the terminal for a waypoint that is not the local node, the waypoint is *popped* and the message re-sent toward the next (finally the bare topic id). This is the oldest

self-healing rule in the system — *a dead waypoint always falls through to the topic id and re-seats* — and it is load-bearing: pinned subscriptions to departed relays heal by exactly this mechanism, and any analysis that forgets it will wrongly predict deadlocks.

An external review made exactly this error in July 2026; the trace is in the Root-Management review response.

The topic decision. Every topic-addressed handler first classifies: *handle* (I am `via[0]` and hold the role, or I am the bare-topic terminal and region-permitted), *reroute* (dead or foreign waypoint at my terminal — pop and continue), *forward* (not terminal), or *reject* (terminal but the region lock forbids rooting here; enabled only when `configureRegionLock({enforce:true})`, which is OFF pre-critical-mass).

X Pub/Sub: The Axon Tree

Pub/sub is *routing-only*: every interaction is a routed message; there are no direct pub/sub connections, no root sets, no K-closest fan-out at publish time. One rule generates the whole design: *the peer where a topic-addressed message terminates acts for the topic.*

Roles, natures, and per-topic state

A peer holding any responsibility for a topic keeps a `role`: `{topicId, isRoot, subscribers (map subHex → {since, lastRenewed}), children (set of child-relay ids), cache (ascending by stamp), cacheIds, tombstones, seq, lastTs, replicas, replSig, replLastFull, backupOf, lastReplicaAt, pulledLw, probeTries, probeAt, metricsOn}`. `isRoot` is the root-claim state machine's state and changes *only* through it (§XI); the remaining fields are the convergence plane's per-topic memory (§X, "Convergence") — each is named where its mechanism is described.

A role acts in exactly one of four **natures**. Each nature carries *obligations* (work the node performs while in it) and a named *eviction path* (how the role ends). This table is normative, and its discipline is invariant I-10: standing state without an evictor is a leak; obligations without a live principal are a storm.

Nature	Marked by	Obligations	Evicted by
ROOT	`isRoot`	stamp, beacon, verify, replicate, serve	demotion / idle sweep
CHILD	in parent's `children`	renew upstream, re-fan down once	subscriber loss + idle sweep
BACKUP	`backupOf` set	hold warm copy; subscribe as election standby	principal gone + re-homed + 60 s
HOLDER	hosted / app-subscribed	renew; advertise `hw`	unhost / unsub / TTL

The natures compose (a HOLDER may be ROOT; a BACKUP is also a CHILD while its principal lives), but every write of a role onto a node must identify which nature it creates and therefore which evictor applies. The first production collapse this system suffered was

a nature nobody had modeled — a BACKUP whose principal was already gone, created by a departing node, whose subscribe obligation ran forever with no eviction path in sight (§XIV, I-10).

Admission: a node can say no

The neuromorphic layer has had capacity discipline from the start: a hard synaptome budget (50), a refusal path, and breadth-then-depth spreading under the cap. The axonic layer had none of it — no budget, no refusal, no spreading. David's pointer was exact: "the system does a pretty good job of spreading connections at the neuromorphic layer; we need to do something similar on the axonic side." The bill for the gap arrived on 2026-07-26: nine production relays on three 961 MB droplets, per-relay role counts of 325, 431, 523 and climbing past 720, memory at 86–92%, five of the nine locked out of the bridge — the join-storm spiral, fed by nodes that could not decline the roles drowning them.

Since 4.46.0 every role acquisition passes `canAcceptRole()`: one gate, four reasons, two tiers. **Bridge** is the hard tier and the floor may never override it — a bridge is transport and introduction, nothing else; `host()` was removed from bridges and `sub()` kept rooting anyway, so a soft tier here would reopen that door on a timer. The soft tier: **not-seated** (a node under 90 s old *or* without a routable non-bridge neighbour — not a bare timer, because the locked-out relays sat at 0 open channels and 58 bound, and a timer would hand roles to exactly those nodes the moment it expired), **saturated**, and **paced** (more than 4 new roles this tick). Every reason is a property of the node's *own* state, self-declared and self-limiting: refusal can only make a node hold less, never acquire what it is not closest to. It is not an exception to the address rule.

The floor is mandatory. If every candidate refuses, nobody roots — and both soft reasons can do that fleet-wide. A simultaneous restart puts everyone in grace; a loaded backbone makes everyone saturated; both happened on production the same day. So a soft refusal with no alternative candidate is admitted anyway and logged `admitted-despite`. A grace period that can partition the network is worse than no grace period.

From a count to a measurement. The original ceiling was `MAX_ROLES` = 96, and production disproved it as an instrument: relays ran 184 to 1,182 roles — seven to twelve times the ceiling — because the mandatory floor must admit every terminal role or drop the message. The count carries no information: 96 idle roles is nothing, 96 hot ones may be fatal, and the same count means different things on different hardware. David's question — "can a node measure its abil-

ity to manage the roles it has?" — replaced the count with pressure measured against real protocol deadlines, at the points where obligations actually complete (4.47–4.53). The metric's own first version could not report failure: it stamped every role serviced at the *top* of the tick, before any work, and read zero while a role sat 95 s past its 60 s replication deadline. The stamp moved to completion.

CAVEAT: this surface is not finished. Refusal exists and is fenced; pressure is measured where obligations complete; what is *partial* is the feedback from measured pressure back into admission and shedding. The health scorecard tracks the gap; a reader should treat "a node can say no" as shipped and "a node sheds load before failing" as in progress.

Subscribe: attach, renew, re-home

`sub(topic, {since})` sends `pubsub:sub {topicId, subscriberId, since, hw, via, latest?}`. `since` is the replay floor: a timestamp, `0` ("`since:'all'`" — full retained history), or now (live tail); `latest` additionally folds the newest retained entry into the first delivery regardless of age. `hw` advertises the sender's own cache high-water — the durability hook: a root that is *behind* a reattaching subscriber issues `pullup` and adopts the newer history without re-stamping.

The serving relay *seats* the subscriber, replays the cache delta above `since` (chunked at 96 KB per deliver), replays all live tombstones (unconditionally — a since-gate cannot express "you missed an old deletion"), and answers with a deliver whose `from` field *pins* the subscriber: subsequent renewals go `via` the pin. Renewal is adaptive: 5 s while unpinned or freshly re-pinned, backing off $\times 1.5$ per stable renewal to a 60 s ceiling; any re-pin or upstream death snaps it back to fast. Subscribers are evicted after 180 s without renewal.

The tree: widen before deepen

A relay over capacity (20 direct) *promotes* one in-region leaf to a child relay and hands it a batch of up to 8 others (`pubsub:adopt`); the child subscribes up toward the parent and re-fans deliveries down to its own subscribers exactly once. Only when every direct is already a child does the tree deepen (the newcomer is delegated to the XOR-closest child). Depth grows like $\log_{20}$(subscribers).

Publish: stamped at the root, confirmed by observation

`pub(topic, message, {signWith})` builds the signed envelope and sends `pubsub:pub {topicId, via, json}` toward the topic (via the current root hint when warm). At the root: verify the signature (B-4),

check freshness (C-2), recompute the topicId from the signed descriptor and reject a mismatch, enforce `write:'owner'`, dedup by msgId and tombstone — then **stamp**: `publishTs = max(lastTs+1, now)` (strictly monotonic; the total order) and `seq = ++role.seq` (dense per-topic counter; a gap in delivered `seq` tells a subscriber it missed something). The stamped message is cached, fanned to subscribers, delivered locally, and eagerly replicated to the cohort.

There is no publish acknowledgment, by design: a PUB carries no return address (publisher-location privacy). Confirmation is *observation* — the publisher stops retrying when it sees its own msgId arrive by any path (own delivery, own root ingest, cohort echo). Until then the pending-publish machinery re-sends the same envelope toward the current root hint each tick, bounded by 30 s and 6 tries; idempotent throughout because roots dedup by msgId. Cold publishers (fewer than 8 neighbours) front-load extra re-sends (§XIII) — each send both integrates the newcomer and gets a fresh shot at the true root.

En route, a relay holding a beacon for a closer root *forwards* the PUB rather than deferring to it: the send resolves to a verdict before any local state moves, and only a `consumed` verdict from the named root re-pins anything (§XI, *Writes forward on receipts*). A publish handed to a corpse must teach the sender something; before 4.59.0 it taught nothing and took the sender's routing state with it.

Convergence: one operation, many policies

Everything that moves topic data in this system is *one operation* wearing different policy: sync(a, b, T) — **make two nodes' views of the topic's stamped set converge**. A view is summarized by four numbers already carried on the wire: message count, high-water (`hw`, newest stamp), low-water (`lw`, oldest stamp), and tombstone count. Two views whose summaries match exchange nothing — the universal quench. The transfer rides the existing verbs (`deliver`, `pullup`/`replayup`, `replicate`, `handoff`), and the ingest side is a single function regardless of which verb delivered the bytes: re-verify every author signature (B-4), dedup by msgId, apply tombstones *before* bodies (I-8), never re-stamp (a timestamp once assigned is never reassigned), and yield to the event loop every 16 messages so bulk adoption cannot starve liveness (I-11).

The policies — who syncs with whom, and when:

Four of these policies were bought by named incidents and their reasons are normative:

- **Split-union (lw)** — after a root transition the timeline splits: the new root holds the post-transition half, the demoted heir the pre-

Policy	Between	Trigger	Moves	Quench / bound
fan-out	root → subscribers	on stamp	the new message	exactly-once dedup
replay-on-seat	relay → subscriber	seat / renewal	since-floor delta + all live dels	since floor
replay-up (hw)	child → root	SUB says $hw >$ mine	the newer delta	hw catches up
split-union (lw)	child → root	SUB says $lw <$ mine	child's full range	once per (child, lw)
empty-root probe	cohort+path → root	root born empty	any history	first ingest; $\leq$3 tries
cohort replication	root → 2 closest	change / new member / 60 s	full state, else *keepalive*	signature match
handoff	leaver → heir	`leave()`	full state	ACK; 2 rounds + 1 fallback
union-at-root	root ← any root	REPLICATE arrives	the sender's state	idempotent merge
read-repair	cohort → stuck subscriber	renewals stall, holder degraded	since-floor delta	first ingest
pull	replica → reader	on demand	one message	one answer

transition half, invisible to the hw rule because it sits entirely *below* the new root's high-water. The child advertises its low-water; a root seeing older history pulls the full range and unions it. *One-shot per (child, lw)*: a refused pull (the child's oldest is tombstoned here, or beyond retention) must not re-fire on every renewal — unguarded, this re-pulled a full cache every 5 s and storm-flapped the testnet relays (4.22.0).

- **Empty-root probe** — a cold subscriber whose SUB terminates at itself becomes a root with an *empty* cache while a live holder one hop away has everything; nothing reliably tells the holder about the new closer root, so the emptiness is sticky. The fix inverts the flow: an empty root *pulls* (`pullup sinceHw:0`) from its cohort and its own lookup path (the runner-up closest is usually the prior holder), bounded at 3 tries and quenched by the first ingest. This was 82% of the field read-misses (4.24.0).

- **Delta-gated replication** — the cohort push sends full state only when the summary signature changed, a new cohort member needs seeding, or a 60 s anti-entropy backstop elapses; every other tick sends an *empty keepalive* that refreshes the backup's liveness clock and nothing else. The earlier behavior — full cache+tombstones to the whole cohort *every tick* — reads as cheap idempotent anti-entropy and is exactly what this document used to claim; under a role-bloat storm it was the bandwidth fuel that collapsed the backbone (4.24.1). Convergence is unharmed: any divergence changes the diverged root's own signature and re-arms one full push.

- **Acknowledged handoff** — §XII.

The principal-liveness rule (normative; the generalization the collapse paid for):

> Standing state on another node may be planted only by a principal alive to maintain it. `replicate` creates a durable relationship — the receiver becomes a BACKUP whose obligations run until evicted — so *a departing node never sends `replicate`*: it transfers ownership (`handoff`) or does nothing. Every new sync policy must state which

nature (§above) it creates on the receiving node and which eviction path applies; a policy that cannot name both is rejected in review.

Replayed stamps more than 5 minutes in the future are dropped (the bad-clock rule). Caches are bounded (1024 messages / 16 MB per role) and expire at 24 h from stamp. On root churn, *election is just subscription*: backups renew toward the topic every tick like any child, so the closest backup's renewal terminates at itself and it promotes with the history already in hand — the same probe-protected machinery every subscriber uses, never a bespoke local vote.

Kill: creator-only retraction

`kill(topic, msgId)` routes the signed kill like a publish (same hint, same pending-retry). The root verifies the kill's own signature, then authorship: if the target is cached, `signerPubkey` must match the target's author *now*; if not cached (kill raced its target), the kill is accepted *provisionally* — the tombstone records the signer, and when the target arrives, a mismatch *revokes* the tombstone and accepts the message. Kills are stamped like publishes (they occupy a `seq` slot), fanned as `del` markers, replicated eagerly to the cohort, and replayed on every renewal while live. A subscriber that never received the body gets no retraction callback (nothing to retract); the tombstone still converges.

Pull, metrics, host

`pull(topic, msgId?)` is a one-shot read answered by the *first replica* the routed request reaches (by-msgId is exact by content address; pull-latest is "recent, not necessarily newest" — hot read paths spread across the cohort instead of hammering the root). A pull that returns nothing says *which* nothing: a responder answered and holds nothing, nobody answered in time, or a reply arrived and would not parse. These used to collapse into one null, and every consumer above manufactured a confident negative from it — one such null cost a full day of misdiagnosis against a channel that was serving 19 subscribers throughout (4.55.0). Three different facts deserve three different values. `metrics` are demand-driven: a `metricson` routed like a SUB arms a renewable 70 s lease at the root, which then publishes signed snapshots `{topic, ts, by, current_count, seq, subscribers, bytes}` to the derived topic `metricTopic(T)` every 20 s while the lease is fresh — no lease, no load; leases survive root promotion via path flags. `host(topic?)` makes infrastructure carry without consuming: a hosted topic is renewed like a subscription (so the node can root it and its cache is durable) but nothing is delivered to the application;

no-arg `host()` volunteers for the node's whole keyspace neighbourhood (retaining any role it wins as terminal) — the relay fleet's default mode.

XI Root Management

The root is emergent — the live peer XOR-closest to the topic id, discovered by routing — so root management is a *convergence* protocol, not an election: wrong claims must yield to right ones, quickly, without flapping, and never to a node that is farther, dead, or departing. Since kernel 4.20.0 every `isRoot` transition passes through one state machine (`rootClaim.js`) with one structured log line per flip.

The companion note *Root Management — the RootClaim state machine* gives the full treatment with worked examples; this section is the normative summary.

Transitions

Transition	why-code	Trigger
become	`sub/pub/kill/metricson-terminal`	topic message terminated here, no role
become	`handoff-heir`	departing root handed us history
promote	`terminal-promote`	non-root relay became bare-topic terminal
demote	`defer-terminal`	stranded traffic deferred to beaconed root
demote	`beacon-closer`	beacon proved strictly-closer live root
demote	`verify-closer`	self-verification found closer terminus
demote	`handoff-better-heir`	post-handoff closer live root ($\neq$ leaver)
claim	`reachable-fallback`	deferral unconfirmed 6 s; self closest reachable
adopt	`adopted-child`	ADOPT made us a non-root child

Every demotion does three things atomically as policy: flip `isRoot`, pin upstream to the winner, and send a confirming subscribe (the winner must *adopt* us or our subtree starves).

The defer gate (never yield to farther, ghost, or leaver)

Before any claim, `liveCloserRoot(topic)` consults the cached root beacon for the topic. Rules, in order: no fresh beacon $\rightarrow$ claim; beacon names self or a node *not strictly closer* $\rightarrow$ claim (the cardinal rule — also the security property: a lying beacon cannot divert traffic outward); otherwise liveness evidence is required, strongest first: a `verified` pointer (network-confirmed by iterative lookup, TTL 90 s), a channel-verified direct neighbour (opens *instantly* on churn — the dead root's channel drop is the gate), or a beacon heard within $1.5\times$`BEACON_MS` $= 30$ s (a live root re-beacons every 20 s; silence is death). The freshness cut applies to *every* form of beacon evidence, verified pointers included. It did not always: until 4.59.0 a `verified` pointer bypassed the cut, so one stale pointer to a dead root could steer traffic at a corpse for as long as the pointer lived. SUB, METRICSON, promotion, and post-handoff use the strict mode: seating infrastructure demands channel or network evidence.

Writes forward on receipts, not on faith

Reads and writes used to pass different gates, and the asymmetry cost a production outage. On 2026-08-02 a host restart took out most of a relay fleet; reads kept working for the surviving holders, because `_onSub` demanded channel evidence before deferring — but `_onPub` and `_onKill` deferred to any beaconed root with no liveness test at all. One stale beacon ate every write to its slice for hours while six of the eight holders sat alive and willing. The read path checked its evidence; the write path trusted a pointer.

Since 4.59.0 a PUB or KILL toward a beaconed root is a *forward*, not a defer, and the distinction is load-bearing:

- **Dispatch first, mutate on receipt.** The old defer demoted the local role and re-pinned upstream *before* sending — so a publish toward a dead relay both vanished and left the sender wired to the corpse. Now nothing about local role or upstream state moves until the dispatch verdict resolves `consumed` *and* `atNode` names the root the forward was aimed at. A failed send moves nothing; the sender's own retries then route by the ordinary rules.
- **A failed verdict revokes only what it tested.** A forward that fails invalidates the one beacon record it was built on — matched by root identity *and* capture time — and nothing else. A verdict that captured no record has no deletion authority: it describes a probe of a pointer the node never held, and a newer-generation beacon that arrived mid-flight survives it (4.59.2). Without the strict match, one slow failure could erase the fresh pointer that had just replaced the stale one.

Both halves are fenced (`fence_pub_defers_to_corpse`, red against the pre-fix kernel, green after), and measured live: on the testnet fleet, writes issued 5 seconds after a root's SIGKILL — inside the freshness window, the exact regime that ate the 2026-08-02 writes — delivered 30 of 30, in each of two runs against different victims. Writes at the 30 s boundary the criterion actually demands: also 30 of 30.

The evidence planes

Beacons: every root advertises `{root, topics[]}` to its 6 XOR-closest neighbours, forwarded 2 layers (basin ≈ 42), every 20 s and immediately on becoming root; pointer TTL 50 s; receivers apply verify-don't-trust (accept only if at least as close as the best locally-known node). A wrong claimant hearing a strictly-closer beacon demotes at once — which also silences its own poisoning beacons. **Self-verification**: beacons only reach the basin, so every root re-checks its own claim with the same iterative lookup subscribers use

— at 6 s after forming (the fresh-topic race window), then every 45 s, batched 3 per tick, never awaited inside the tick. Finding a strictly-closer live node seeds a `verified` pointer and demotes. This whole plane depends on the DHT adapter returning a real `LookupResult`: the default adapter used to return a bare id while every consumer read `r.path`, so self-verification and the iterative escapes were silent no-ops on *every* standalone peer — present in the code, absent from the network — until 4.28.1. **Death sweeps**: when a peer's channel closes or it announces leave, every beacon naming it *and every upstream pin on it* is purged, with the pinned subscription's renewal clock reset — re-homing is next-tick, and the reachable-root fallback re-arms. **The replica ledger**: `role.replicas` is durability bookkeeping — it decides whether a topic counts as safely replicated and whether history survives the root leaving — and it records a backup only on a *verified dispatch*: the replicate resolved `consumed` at the named node. It used to record on the strength of a local call that had not thrown, and routing does not throw — it reports failure by resolving. Measured before the 4.57.0 fix: thirteen replicate sends, all failed, ledger reading `replicas: 2`. A ledger written from intentions is not a ledger.

The fallback asymmetry

A subscriber unpinned for 6 s despite renewals (its hint names a closer-but-unreachable node) claims the root itself if it is closest among *reachable* peers: a reachable root beats a closer unconfirmed one. Claiming wrongly is cheap — beacons or verification demote it within one period; deferring wrongly strands the topic forever. When the mesh is bare (no non-bridge neighbours), a self-SUB terminal claims nothing — "terminal at self" on an unmeshed node is isolation, not closeness; the seat is held and the fast renewal re-runs the decision once meshed. Publish-side is not gated, by design: a solo node still roots its own publish.

XII Lifecycle: Join and Leave

Join: `connect({bridge, identity?})` performs the bridge bootstrap of §VIII, resolves the directory, *self-integrates*, and returns a started peer. There is no second call to make and no hidden step — `connect()` is the single, complete entry point (4.40.0), and the completeness was bought the hard way. Self-integration — `findKClosest(self)` plus opened, authenticated channels to the results, the primitive that lifts a newcomer from the passive-adoption floor (sim-validated: 7–27% reachability without it, 95–98% with) — used to run only on the sponsored `join(sponsor)` path. The two

one-call paths every real application uses, `connect()` and no-sponsor `join()`, both skipped it (4.39.0). So applications sat at the churn floor and self-rooted their topics as singletons in sparse regions: fresh subscribers read nothing, publishers stranded on leave — and the one application that worked, worked only because it called the integration primitive by hand. An entry point that requires a manual step to work is not an entry point. Reachability still ripens over the first seconds as inbound traffic writes the newcomer into neighbours' tables; the cold publisher burst and the fast renewal floor are shaped around exactly this window.

Leave (`leave({timeoutMs})`) is a contract with three promises, each bought by an incident — and the *order* of the promises is itself the contract. `leave()` used to announce the departure first. Receivers hard-close the leaver's channels on that signal (the proactive re-anchor fast path), so every routed `handoff` that followed found no route, terminated back at the sender, and was silently discarded — captured live as 173 of 173 leave handoffs boomeranging with zero acks. Every topic whose only copy rode a handoff died with the leaver, deterministically, about 10% of publishes. The courtesy call announced the death before the will was read. The order is now drain, hand off, *then* notify (4.32.0), and no promise below may be reordered past another:

1. *Drain on evidence*: poll until the pending publish/kill maps are empty — a confirmed publish departs instantly, an unconfirmed one gets the caller's full window (never a hidden cap). Internal waits are ref'd timers: Node must not exit mid-leave.

2. *Hand off every last copy, confirmed*: for each cache-bearing topic the leaver ROOTS *or holds as a BACKUP*, resolve an heir *and a runner-up* — in-region first (an out-of-region holder is durable but unfindable by routed reads), local K-closest, falling back to the *iterative* lookup when the leaver's table is thin (the fresh burst-publisher case) — with 8 resolutions in flight, singleton roots first, the whole handoff bounded in proportion to the role count (cap 60 s). Rooted-only handoff was the HANDOFF_GAP: under churn a copy cascades root → backup, the backup departs, and the last copy dies with a node that never considered it worth handing off — 100% of a field-measured 9–13% restart loss classified as exactly this (4.31.0). Then send `pubsub:handoff {topicId, msgs, dels, from}` in *acknowledged rounds*: all unacked heirs per round, a shared 700 ms ack window scaled under load, two rounds. The heir replies `handoffack` — and the ack claims only what the heir *holds*. The ingest path had five silent early-returns, and the ack said `{topicId}` regardless, so an heir that rejected half a batch acked byte-identically to one that took all of it; the leaver then ex-

empted the topic from every retry and departed with the rejected half (4.45.0). Fire-and-forget was worse still: it dropped a topic's last copy whenever the one send didn't land (40/40 confirmation failures in the field). A topic still unacked after the rounds gets *one* fallback `handoff` to the runner-up — worst case two proper holders whose caches union — and *never* a `replicate`: acks are a race against load, so "unacked" usually means "acked late," and a departing node spraying replica state plants backups of a dead principal (the principal-liveness rule; violating it collapsed the backbone twice). `from` names the leaver so the heir purges the leaver's ghost beacon and never defers its new claim back to it.

3. *Then silence*: notify peers in parallel (they run the death sweep immediately), stop the tick, clear all retry state. A peer that has left sends nothing — and the network stops listening for it.

Abrupt death is the same story without the courtesy: channels drop, neighbours sweep beacons and pins instantly, the warm cohort holds the history, and election-by-subscription seats the successor.

XIII The Repair Plane and Timing Model

One scheduler (`refreshTick`, every 5 s) drives all periodic repair: subscription renewals (adaptive), hosted re-announces, backup renewals (every tick — infrastructure never backs off), the pending publish/kill retry, cohort replication, metrics leases and snapshots, subscriber eviction and cache/tombstone expiry, role teardown, beacon emission, and the self-verification batch. The only timers outside the tick are the cold-publish burst and the one-shot first-publish re-send — all tracked and cleared on stop/leave.

Constant	Value	Reason
tick	5 s	must be $\leq$ the renewal floor
`RENEW_FAST_MS` / `RENEW_MS`	5 s / 60 s	orphan window vs. steady traffic ($\times$1.5 backoff)
`DROP_MS`	180 s	subscriber eviction ($\geq 3\times$ ceiling)
`ROOT_CLAIM_MS`	6 s	unconfirmed-deferral window ($\geq$2 ticks)
`BEACON_MS` / TTL	20 s / 50 s	advert cadence; $1.5\times$ = corpse-freshness cut
`BEACON_FANOUT` / layers	6 / 2	basin $\approx$ 42 nodes
verify first / steady / batch	6 s / 45 s / 3	fresh-topic race; steady re-check; no lookup storm
`ROOT_REPLICAS`	2	cohort size (singleton-root durability)
replicate full-push backstop	60 s	delta gate: keepalives between; anti-entropy floor
replicate full-push budget	32/tick	join-storm pacing: seed a newcomer over ticks, not in one (I-11)
ingest queue / slice	4096 / 8 ms	bulk verify can't monopolize the loop; overflow drops newest (logged)
mesh re-warm	$<$3 peers $\times$ 3 ticks, 60 s cooldown	a dissolved mesh re-runs self-integration
handoff ack window / rounds	700 ms base (load-scaled) / 2	confirmed departure; whole handoff $\propto$ roles, cap 60 s
backup eviction (re-homed idle)	60 s	bounds set growth only; never fires while electable
empty-root probe	800 ms, 5 s, $\leq$3, fan 4	delay / re-try / cap / fanout; quench on ingest
pending TTL / tries	30 s / 6	bounded retry; loss^7 negligible at 30%
cold burst	5×200 ms $+ 5 \times 400$ ms	integrate-and-retry while the table warms
first-publish re-send	200 ms	catch a tree formed microseconds before
`MAX_DIRECT` / batch	20 / 8	delegation threshold / handoff batch
cache	1024 msgs / 16 MB / 24 h	bounded hold, keyed on root stamp
metrics lease / cadence	70 s / 20 s	demand-driven; self-expiring
future tolerance	5 min	bad-clock rule for replayed stamps

Convergence intuition: a wrong claim inside the beacon basin corrects within ≤20 s; outside it, ≤6 s fresh / ≤45 s steady; a deferral to an unreachable node resolves in ∼6 s; a dead root's influence ends at channel-close for neighbours and ≤30 s otherwise — and since 4.59.0 that bound governs *writes* as well as reads, because both pass the same gate (I-12); a subscriber whose neighbour upstream dies rehomes next tick; a backed-off subscriber heals no later than its next renewal via the waypoint-pop rule.

XIV Invariants

Normative, and this section is the authority. An implementation that violates one is wrong even if it benchmarks well.

Each entry names the regression test that enforces it. Where no test exists the entry says **UNFENCED** — an invariant nobody checks is a wish, and marking it is more useful than implying coverage that is not there.

On numbering. Three files carried invariants under three schemes: `I-1..I-14` here, `S1..S6` and `B1..B13` in an `INVARIANTS.md` beside this file, and a third set in the kernel repo. They overlapped without cross-referencing, so a reader could not tell whether `B3` and `I-2` were two rules or one. They are one. This section supersedes both files; the `B` column below preserves the old identifier so existing references still resolve.

Structural rules — how the code is built

These constrain the shape of an implementation rather than its behaviour. They are here because every one was extracted from a defect that recurred until the structure changed.

#	Rule	Fence
S1	One transition site per state. Every change to authoritative state goes through a single function emitting a single log line.	`smoke_root_claim`
S2	Derive, never store, what can drift. Role natures are computed from ground truth, never persisted — this killed the #333 drift class structurally.	`smoke_role_natures`
S3	One operation, a typed table, a CI-enforced boundary. N bespoke paths collapse into one parametrized operation behind a complete table, one emission site per wire verb, and both properties are tests.	`smoke_sync_engine` (table complete, closed at 8 rows) · `smoke_emission_sites`
S4	Closed shapes. A state object declares every field it can ever carry in its one constructor; no runtime graft-ons.	**UNFENCED** — needs a shape-freeze test
S5	Budgets scale with work. Any window bounding $O(K)$ work scales with K; prefer progress-aware exits over time. Flat constants against variable batches are the mass-leaver bug class.	`smoke_handoff_scaling`
S6	Orchestration under one clock. Periodic work is named, ordered, individually testable units under ONE tick — never N independent timers, because timer multiplication changes interleaving semantics.	**UNFENCED** — the gate for the `refreshTick` refactor

Behavioural invariants — what must always hold

Each entry closes with its fence and, where one exists, the identifier it carried in `INVARIANTS.md`.

I-1 A topic *converges* to a single root, and wrong claims converge without flapping. This is a liveness property, not mutual exclusion: duplicate roots are an expected transient, and the system contains no distributed lock that would forbid them. Two bounds qualify it. Reconciliation reaches only as far as `rootReplicas` (currently 2),

so a second root outside that reach is not repaired by reconciliation and persists until churn removes it. And under partition each side converges independently and stamps its own dense sequence; the merge on heal is a union, not an ordering. An implementation must converge and must not flap; it must not assume an exclusivity it does not have. Three sub-rules carry it: a joiner iteratively verifies past its local table before self-rooting (*was B2*); a root union-ingests a `REPLICATE` rather than usurping or refusing it (*was B4*); and an empty self-root pulls cohort history before acting as root (*was B9*).

Fence: `smoke_root_hint` · `smoke_interloper_convergence` · `smoke_split_history_union` · `smoke_empty_root_pull`

I-2 Never defer a root claim to a farther node, a ghost, or the node handing off.

Fence: `smoke_leave_handoff_burst` (heir no-defer case) · *was B3*

I-3 A recovery path never waits unboundedly on — or dies on — the failure it handles.

UNFENCED. Issue #339 — `transport.start()` must fail loud.

I-4 A peer that has left is silent, and its rooted history departs before it does. Handoff completes *before* notify — data first, funeral announcement second (*was B5*) — a departing non-root holder hands off unless the root is positively alive, since passive freshness lies during mass teardown (*was B6*), and a mass leaver's sole-copy topics go first, singletons before replicated roots before holders (*was B11*).

Fence: `smoke_leave_teardown` · `smoke_pubsub_leave_handoff` · `smoke_backup_handoff` · `smoke_handoff_scaling` (tier order only indirectly — **partial**)

I-5 A client is never judged by time the server wasn't listening.

UNFENCED at the kernel; the bridge's hello-timeout second strike is issue #338.

I-6 Observability surfaces exist or fail loudly — never silently zero.

UNFENCED. Metrics were silently dead across all of 4.x before anyone noticed; a metric name that does not match its referent is the same defect (`axona_status` calls the synaptome `mesh`, issue #411). No operator surface exposes live channel count, so "did the mesh survive?" cannot be answered from outside.

I-7 Fixes must hold for 100%-transient peers: no mechanism may privilege stable nodes to mask churn.

UNFENCED, and unfenceable by a unit test — this one is enforced at review. Stability-weighted root election was proposed, simulated, and rejected under it.

I-8 Migrated cache never resurrects a killed message: every migration path carries tombstones, applied first. A kill callback fires only if the body reached that app (*was B8*).
Fence: `smoke_kill_migration` · `smoke_pubsub_kill` · *was B7*

I-9 Publish confirmation is observation, not acknowledgment; nothing may add an ack channel that discloses the publisher's location.
UNFENCED behaviourally; enforced by the envelope's shape — it names a signer and carries no node id and no region. Two attempts to add one were reverted.

I-10 Standing state is bounded by demand, never by churn history: per-topic state anywhere is $O(\text{subscribers} + \text{cohort})$; no mechanism may create state that accumulates with join/leave *events*; every write of standing state onto another node names its eviction path in the same change. Corollary: the principal-liveness rule (§X) — a planted nature is retired only when its principal is gone *and* re-homed.
Fence: `smoke_sync_engine` (evictor column complete for every policy row) · *was B10*

I-11 Bulk work never starves liveness: unbounded batch ingest/emission yields to the event loop at a fixed stride; a node is never evicted by its peers *because* it was absorbing history.
UNFENCED as a property; the pacing constants are in the timing table (§XIII) and the join-storm that motivated it is issue #332.

I-12 A write is never handed to a root without live evidence — a channel, a network-verified pointer, or a beacon fresher than 1.5×`BEACON_MS` — and local routing state mutates only on a `consumed` verdict from the node the write was aimed at. The read path and the write path pass the same gate.
UNFENCED on the write path — issue #422 records a `PUB` deferring to an unreachable root. The read side is covered by `smoke_ghost_read`.

I-13 Durable bookkeeping is written from receipts, never from sends. A replica, an upstream pin, a beacon invalidation — each requires a verdict naming the node it concerns; a send that merely didn't throw proves nothing, because routing reports failure by resolving.
UNFENCED. The general shape — a send whose non-throw was read as delivery — has appeared three times.

I-14 A negative result names its kind. Timeout, answered-empty, and unparseable are three facts; any surface that collapses them into one value invites its caller to manufacture a confident wrong conclusion.

UNFENCED. Issue #418 is a live instance: a one-shot subscribe reads zero on a topic a standing watch is holding traffic on, and zero is indistinguishable from empty.

I-15 Transport identity is ephemeral; author identity is durable. A node's `nodeId` and its keypair are minted fresh every process start and written to no persistent store — not a namespace, not a file, not a snapshot, not a container volume. An author identity may and must persist. This one protects people rather than correctness: a `nodeId` surviving restart is a durable correlator linking a node's sessions to each other, to an IP, and so to a person, and the countervailing benefit is nil, because a node returning under its old id gains nothing from a mesh that has already healed around its absence. Reconnection continuity, route retention, log correlation, deduplicating concurrent processes, and node-keyed reputation are each a signal the design is wrong, not a justification.

Fence: `smoke_persistence_wiring` (restart ⇒ different `nodeId`, author survives) · `smoke_snapshot` · `fence_transport_identity` (static, one per repo) · *was I-ID*

I-16 Region is an optimization, never a wall. A region prefix biases *where* a topic prefers to live; it never decides whether a node may hold, root, or route one, and a sparse region must roll over into its neighbours. The failure mode of a missed optimization is a slower hop; the failure mode of an enforced wall is data with nowhere to live. The refusal machinery exists and is dormant behind `isRegionLockEnforced === false` — *leaving it off is the invariant*, which is the opposite of what a reader would conclude from the flag's existence. Ranking, mint-point choice, and holder preference may depend on region; admission and capability may not.

Fence: `smoke_region_lock` covers the dormant path only — **no fence asserts the default stays off**. Treating the prefix as a wall once produced 0% cross-region delivery (4.17.1). · *was B1*

I-17 A bridge is a bridge — it transports and introduces, and holds no topic role at any load, in any topology, with no floor exception. Three independent doors enforce it: routing skips it in the next-hop scan, it does not `host()`, and `neverRoot` makes its own `sub()` unable to self-root at a tier the admission floor may not override. The tempting exception — let it root when it is the only candidate — would have it root exactly when it can least afford to, and buys only a few seconds of empty-network durability during a fresh launch.

Fence: `smoke_role_admission` (refusal is HARD; the floor must not override) — **no fence asserts a bridge peer ends bring-up with `axonRoles` empty**. · *was B12*

I-18 Capacity is measured, never counted. Fitness to hold roles is observed pressure against real protocol deadlines — service staleness against `DROP_MS`, tick lag against `HELLO_DEADLINE_MS` — never `axonRoles.size`. A count is inventory; pressure is capability, and capability is what predicts failure. A node holding the maximum roles and servicing all of them on time is a healthy node.

Fence: `smoke_role_admission` · *was B13*. Caveat in the same breath: `servicePressure` once sat pinned at 0.028 while a node held 546 roles at 200% CPU, so the measure is right in principle and has been wrong in practice — a metric that does not match its referent is I-6's problem.

Process rules — how changes land

Not properties of the running system, so not numbered with the invariants; they are here because every one was bought with a bad day.

- **Gates are pre-registered, not post-hoc.** The SLO is fresh-subscriber delivery percentage. A measurement is REPS $\geq$ 5 reported as mean $\pm$ sd; a single run is directional only.
- **No behaviour change without a disproof.** A failing repro precedes every fix, and the repro becomes the fence. This is why the UNFENCED entries above are unfenced — they were reasoned to, not reproduced.
- **Mechanism freeze during refactor phases.** Incident response is a revert or a minimal guard carrying an issue ID and a removal date. No new mechanisms.
- **A mixed-version compatibility statement** is required for any change touching wire-adjacent behaviour. Production runs several kernel versions at once — 4.35 peers were observed during 4.41.
- **Read deployed versions from the running service.** Any version used to size, gate, or justify work comes from `/healthz` at the time of writing, never from a note, a memory, or a previous document. A release was once sized as four versions of new subsystem when it was four bugfixes, and the error survived a full review pass because every reader trusted the same note.
- **Verify per claim, not per narrative.** A review checks each numeric claim against source. A plausible story carries false figures through unchallenged.

XV Robustness: What Fails, and What Happens

A subscriber's relay dies. Neighbour case: death sweep drops the pin, renewal snaps fast, re-home next tick. Otherwise: the next renewal routes toward the corpse, pops at the live terminal, re-seats at the true root, and the deliver re-pins — worst case one renewal interval.

The topic-closest node is alive but useless. Dead is the easy case — the channel drops and everything sweeps. A *degraded* holder answers the transport and fails the protocol, so nothing sweeps and a subscriber can renew into it forever. Read escalation (4.33.0) probes past dead *and* degraded holders, and stuck-subscriber read-repair (4.36.0) lets the cohort push the since-floor delta to a subscriber whose renewals stall against a holder that will not serve — quenched on first ingest, one row in the policy table. The A/B under packet loss is the receipt: 4.36 against 4.35, same loss rate, read-repair carried the difference. The bridge departure hints that shipped alongside are marked TEMPORARY in the source and carry a reachability guard; a permanent mechanism must not lean on the bridge (§VIII).

The root dies abruptly. Its beacons are swept by neighbours; the warm cohort already holds cache+tombstones; the closest backup's next renewal terminates at itself and it promotes, gap-free; the rest re-home under it. Singleton topics survive because replication is proactive, not reactive. Writes in flight ride the forward contract: a send at the corpse fails to a verdict, the sender's pending retry re-routes, and the stale beacon is cut at 1.5× the beacon period even if nothing sweeps it sooner. Measured on the live fleet: 30 of 30 writes issued 5 s after a SIGKILL delivered, twice, against different victims.

A stale beacon names a corpse. The pre-4.59 failure mode, and the one that cost 2026-08-02's production writes: the pointer outlives the node, and everything routed by it vanishes while the node's neighbours still answer reads. Now every beacon path — verified pointers included — carries the 30 s freshness cut, a forward that fails revokes exactly the pointer it tested, and no verdict deletes a pointer it never captured. The corpse's influence over writes ends with its beacon's freshness, not with luck.

A send fails without a sound. Routing does not throw — `routeMessage` reports failure by *resolving* `{consumed: false}` — so a caller that discards the promise cannot tell delivery from silence, and no try/catch will ever tell it. The containment lives in one place: every emission path returns the verdict, a transport

error becomes a failure verdict of the same shape, and a rejecting transport cannot kill the process (4.57.1). Callers that ignore the result stay safe; callers that read it get the truth. The 4.49.0 class — a node dead in every duty while its health endpoint reads fine, bought by one swallowed `TypeError` in the repair tick — is fenced against recurrence.

The root leaves gracefully. Handoff pushes history to heirs (parallel, timeout-raced, thin-table lookup fallback); heirs never defer back to the leaver.

A publish strands. The greedy walk hit a local minimum; the pending retry re-sends toward the refreshed hint each tick (bounded); cold publishers burst. The retrying is idempotent end-to-end.

A cold reader becomes an empty root. Its SUB terminated at itself while a live holder kept the history — the sticky read-miss class. The birth probe pulls from the cohort and the lookup path within a second; the tick re-probes while empty, bounded at 3; the first ingest quenches. The reader serves history, not emptiness.

A burst publisher churns out under load. Its `leave()` hands off in acknowledged rounds; acks lost to load trigger one fallback handoff to the runner-up — never a replica spray. Worst case two holders whose caches union; the fleet's role population is unchanged (I-10).

History arrives in bulk. A joiner adopting thousands of cached messages yields to its event loop every 16; heartbeats interleave; its mesh peers do not evict it for absorbing what they sent it (I-11).

Two roots exist transiently. Near-miss terminals can claim while evidence is thin — by design. Beacons reconcile within the basin; self-verification reconciles across it; cohort anti-entropy keeps the data unified while the claims converge; a since-all rejoin reads the union.

The network partitions. Each side converges internally (roots, cohorts). On heal, beacons and verification collapse duplicate roots; anti-entropy unions the caches; per-publisher `seq` lets readers order interleaved histories.

The bridge restarts. Existing mesh connections are unaffected (the bridge is signaling-only). Reconnecting peers ride through transient 502s — the reconnect path survives the very errors reconnection produces (I-3) — and re-establish. Nothing about topic state lives on the bridge.

A peer lies. A forged envelope fails B-4 verification at the root; a forged kill fails authorship and is revoked even if provisionally recorded; a lying beacon cannot point farther than the receiver's best-known node; gossip cannot write routing tables; a claimed nodeId must hash from the proven key.

Clocks are wrong. Stamps are root-monotonic regardless of wall clocks; replayed stamps too far ahead are dropped; freshness gates use bounded skew.

XVI The Ship of Theseus: Durability Under Replacement

Replace every plank of a ship, one at a time, and ask whether the same ship remains. Replace every node of an Axona network, one at a time, and the question stops being philosophy: does the data survive? The preceding section lists the mechanisms; this one reports what happens when a 200-node fleet is replaced end to end, node by node, with the mechanisms doing their work. The numbers are from the theseus harness (dht-sim, 200 nodes, 32 topics per run, 10 messages per topic, replacement interval as the independent variable); single runs are noise, so every figure is aggregated across repetitions.

Loss comes in whole topics. Across every arm measured — soft and hard, thousands of topics — a topic read back after full replacement is either complete or empty, never partial. The cohort replicates full state; a survivor has everything or was never in the cohort at all. This granularity is the fact the rest of the model hangs on, and it is why the durability question reduces to one event: does a topic's last warm holder die before anyone notices.

Graceful replacement loses nothing. With `leave()` — handoff before announcement, §XII — loss was 0.000% across 180 runs, 1,890 topics, 60,165 messages, at every replacement speed measured. The Ship of Theseus sails home intact, provided each plank is removed by a carpenter and not by cannon fire.

Abrupt replacement is a race, and the clock is the killer. Kill nodes with no handoff — SIGKILL, the cannon — and loss appears, governed by timing rather than placement: 2.2% of topics at a 1.2 s replacement interval, falling roughly 30-fold as the interval stretches 16-fold (log-log slope -1.28), toward zero as replacement slows past the cohort's own re-check pace. Nodes died in random order, so the victims scattered across the address space; kill in address order and a topic can lose its whole cohort at once. 2.2% is the good case.

Holder count is a proxy, not a cause. Per-death, across 904 single-death topics: one warm backup lost 25.0%, two lost 8.9%

(±2.3, n=616), three lost 0% (n=202). Read alone, that table says "add a replica." The controlled comparison says otherwise: raising `ROOT_REPLICAS` from 2 to 3 roughly halved loss at the cost of 50% more replication traffic — and installed no floor. Worse for the simple story, topics holding three warm backups in the 3-replica arm still lost 11.2% under identical churn while the 2-replica arm's three-backup topics lost nothing. The count was standing in for something else: a *quiet, well-connected neighbourhood* whose members hear about the death in time. Durability is not how many copies exist. It is whether the survivors' next conversation happens before the next death.

Communication is the substrate. That reading was David's conjecture — the network is built around information flow, so a network that talks more survives more — and two measurements back it. First, the renewal asymmetry: backups re-check their principal every tick, ungated at 5 s, while ordinary subscribers back off from 5 s to 60 s; the cohort talks twelve times as often as the subscriber population, and the cohort is what survives. Second, directed integration: a newcomer becomes reachable only when traffic arrives *at* it, writing it into its neighbours' tables — about 4 directed packets per newcomer achieved what two and a half times as much undirected warmup traffic did. The lever on durability is the rate of the right conversations, not the size of the crew.

The live fleet agrees, with a tail. On the 26-relay testnet fleet (kernel 4.59.2), the census reads a median of three warm holders per topic — root plus two backups, as configured — but 2 of 30 freshly-minted topics sat at two holders before any churn at all, and the tail widened to 6 of 30 under sequential kills. Six targeted deaths (the largest role-holders, 90 s apart) lost nothing: 30 of 30 topics replayed complete. Six deaths cannot observe a per-death rate of a few percent on a thin tail; the sim's 904 deaths could, and did. The thin tail is where the loss lives. Watch it in hours, not minutes.

What this means for operating a fleet: call `leave()` on every shutdown you control; roll one node at a time with the replacement already live; treat abrupt mass death as the one event the protocol does not yet promise to absorb, and measure it before trusting it. The open design limit is recorded rather than hidden: root reconciliation reaches only the `ROOT_REPLICAS` cohort, so a duplicate root that forms beyond it is permanent until re-rooted — a known limit, tracked, not yet paid for.

XVII The Application API

The kernel's public surface on `AxonaPeer` (browser and Node identical): lifecycle — `connect()` / `join()`, `start()`, `leave({timeoutMs})`, `stop()`, `ready()`; pub/sub — `pub(topic, message, {signWith})`, `sub(topic, {since, replayLatest})`, `unsub`, `kill(topic, msgId)`, `pull(topic, msgId?)`, `metrics(topic)`, `host(topic?)`, `unhost`; identity — `createNodeIdentity`, `createAuthorIdentity` (module exports); direct messaging — `send`/`notify`/`onMessage`; introspection — `health()` (roles, hosting, mesh truth), `peers()`, `rootedTopics()`, `onPeerJoin`/`onPeerLeave`, `onLog`/`onError`, `onUpgradeRequired`; persistence — `snapshot()`/`fromSnapshot()` plus the PersistenceAdapter (IndexedDB, file, memory). Topic arguments are descriptors (§VI); `signWith` takes an author identity or the `ANONYMOUS` sentinel. Applications render message bodies through `std/message` and move bulk bytes through `std/chunk` (10 KB chunks under the 15 KB reliable floor; chunked receive returns per-file msgIds so a transfer can later be `kill`ed).

Two return-shape facts, each the residue of a confident-false-negative: `pull` returns the *full envelope* — msgId, stamp, signer, body — not the bare body; the bare-body shape made every publish confirmation built on it read `null` and fail forever (4.29.0). And `metrics().publishes` is a real throughput counter; it was wired to nothing and reported zero for the life of the 4.x line until 4.41.0 — an observability surface that existed and silently lied, the exact class I-6 forbids.

XVIII Security Model

Self-authentication end to end: content believed only against the author key it carries (B-4 at every root ingress and every stamped re-ingest); transport believed only against the node key proven on the channel (axona/5, DTLS-fingerprint binding on WebRTC); freshness and per-publisher monotonic `seq` against replay (C-2); domain-tagged signatures everywhere (no cross-protocol replay); total canonical encoding shared by signer and verifier (C-1); inbound size and count caps on every attacker-controllable field (D-1: 256 KB hard publish ceiling, 15 KB reliable floor, via ≤ 8, beacon topics ≤ 256); first-party-only routing tables against eclipse (B-3); verify-don't-trust on every hint (beacons, pull answers, replayed history). Address grinding is answered by memory-hard proof-of-work on a *separate puzzle* $H(\text{domain} \,\|\, \text{role} \,\|\, \text{pubkey} \,\|\, \text{nonce})$ — never on the address itself, which would skew the keyspace (calibration complete; activation gated). The standing privacy rule bears repeating: the protocol learns *who* from

envelopes and *where* from node ids, and no layer may join the two.

XIX Production Deployment

Two isolated networks, partitioned hermetically by wire major. **Production** runs three kinds of standing infrastructure, and the doc describes their natures rather than their addresses: multiple *bridges* (Dockerized, federated by uplink, advertising on the directory topic) spread across regions; a *relay backbone* — groups of keyspace-hosting relays in several geographic regions, so no single region's outage removes the majority of standing holders; and the reference application, `axona.chat`. **Testnet** is the staging network — its own bridge and apps, hermetically separate — and every kernel release soaks there first; prod promotion is a deliberate, evidence-gated step (full suite, external axonSpec gate, settled-mesh soak acceptance, and the standing runbook). Every component surfaces its `KERNEL_VERSION` at a runtime-inspectable surface; deploys are staggered and judged only after the mesh settle window, never by one-shot fresh clients.

XX Reconstruction Guide

To rebuild a conformant peer from this note alone, implement in this order, testing each layer before the next:

1. **Canonical form and ids** (§VI): total canonical JSON; SHA-256; Ed25519; the 264-bit id discipline (region byte, hex↔BigInt); `deriveTopicId` with its region-resolution rule; `msgId`. Test: same inputs, byte-equal outputs, cross-implementation.

2. **Envelope and kill** (§VI): build / sign / verify with domain tags; freshness and seq rules. Test: a signature from one implementation verifies in the other; domain confusion fails.

3. **Routing** (§VII): synaptome with first-party admission; greedy `routeMessage` with terminal detection and hop budget; iterative `findKClosest`; dead-peer skip; bridge exclusion. Test: terminal correctness on random topologies; lookup escapes a constructed local minimum.

4. **Transport and handshake** (§VIII): any reliable channel + the axona/5 mutual auth with channel binding; the wire-major gate. Sim transport first; real transports are substitutable.

5. **The routed pub/sub core** (§IX, §X): the topic decision (handle / reroute / forward / reject) with waypoint popping; roles; SUB seating + replay + pinning; PUB verify-stamp-cache-fanout;

DELIVER re-fan and re-pin. Test: N subscribers, one root, 100% delivery, total order by stamp.

6. **The root-claim state machine** (§XI): the transitions and the defer gate with its three evidence tiers, as one module with one transition function; then the evidence planes (beacons with verify-don't-trust, self-verification, death sweeps). Test: the decision table directly — farther beacons never defer; ghosts never defer under strict mode; every flip logs once.

7. **The repair plane** (§XIII): one tick driving renewals (adaptive), pending retries, replication, eviction, expiry, beacon/verify cadence; the constants table verbatim; complete teardown on stop.

8. **The convergence plane** (§X, §XII): one sync operation, the policy table verbatim — delta-gated cohort replication with keepalives, replay-up (hw) and split-union (lw) with the one-shot guard, the empty-root probe, kills with provisional authorship, and leave with evidence-drain, *acknowledged* handoff, and silence.

9. **Prove the invariants** (§XIV): write all eighteen as executable tests before tuning anything — including the ones marked UNFENCED, which are where this implementation has been wrong most often. Then characterize on a live mesh — healed delivery after a renewal cycle is the headline number, single runs are noise (REPS ≥ 5), and no verdict comes from a seconds-old client.

Four traps this document exists to prevent. *Forgetting the waypoint-pop rule*: you will wrongly predict deadlocks and "fix" them into churn storms. *Adding a root-claim site instead of a guard in the one decision table*: you will recreate the patch-interaction bug class the 4.20 refactor killed. *Adding a data-movement mechanism instead of a sync-policy row*: every mechanism you add carries its own trigger, quench, and guard, and interacts with all the others — the 4.22–4.24 incident chain, including two backbone collapses, was exactly this trap sprung four times. *Letting a departing or transient node plant standing state*: replicate-from-a-leaver reads as a cheap durability win and is a cascade seed; the principal-liveness rule exists because the cheap win collapsed a fleet.

What you will still have to derive

"From this note alone" is a claim about the *model* and the *wire contract*, and it should be read narrowly. Four things are not here, and a reimplementer will meet all four.

The constants are ours, not derived. Every value in the timing table (§XIII) carries its reason, which is what lets you re-tune it —

but the numbers came from a mesh of tens to low hundreds of nodes on consumer hardware and cloud droplets. At a different scale or a different latency profile they are a starting point, not a specification.

The failure catalogue is partial by construction. The robustness section (§XV) describes failures that have happened to this implementation. Everything in the open issue register is a failure we know about and have not fixed; what neither list contains is the class nobody has hit yet, and there is no reason to think the network has finished producing them.

Nine invariants are marked UNFENCED (§XIV). Those are statements of intent that the code is believed to satisfy. A second implementation that satisfies the fenced ones and violates an unfenced one will not be caught by anything either project runs today.

Interoperability is unproven. No second implementation exists, so every "cross-implementation" test above is a test nobody has run. The first one will find things — most likely in canonicalization, where byte-equality is required and the specification is a paragraph. That is not a caveat on the document so much as the reason to build the second implementation.

XXI Appendix: The Life of a Message

Publisher P (browser, us-east), subscribers $S_{1..12}$, fresh topic T = {region: useast, owner: null, name: 'lobby', write: 'open'}.

1. S_1 subscribes: SUB routes toward T, terminates at R (the closest live peer) → `become('sub-terminal')` — R is now the root, beacons immediately, self-verifies at 6 s. S_1 is seated, pinned by the deliver-from, renews via R.

2. $S_{2..12}$ subscribe; all reach R (beacons steer near-misses). At 21 directs, R promotes S_2 to a child relay with a batch of 8 — the tree widens.

3. P publishes: envelope signed with P's author key; PUB routes toward T (hint-assisted); R verifies, stamps (`publishTs`, `seq=1`), caches, fans to directs and the child (which re-fans), replicates eagerly to its 2-member cohort. P, being subscribed, receives its own message — observation confirms; retries stop.

4. P kills the message: signed kill routes to R; authorship matches; tombstone recorded (`seq=2`), del marker fans down, cohort updated eagerly. A later `since:'all'` joiner replays survivors only.

5. R leaves: drain (nothing pending), handoff of T's cache+tombstones to heir H (`from: R`); H acknowledges (`handoffack`) inside the first

round, so no retry and no fallback fire; notify, silence. H purges R's beacon, keeps the claim, beacons; renewals re-seat everyone under H within one fast cycle. Total order continues above the inherited `lastTs`.

6. Late that night a cold reader C subscribes from a fresh node that happens to sit closer to T than H: C's SUB terminates at itself and it becomes root — empty. The birth probe pulls T's history from the cohort and from H (on C's lookup path) within a second; C serves the full timeline. H re-homes under C as a child; its `lw` advertisement confirms nothing older is stranded. No mechanism special-cased C's newness — the same policies that run everywhere ran here (I-7).

Axona Architecture v4.59.2 — 2026-08-05. Versioned to the kernel release deployed on testnet that it describes; re-version on the release that next changes protocol behavior. The eighteen invariants and six structural rules are §XIV of this document and are not maintained anywhere else. Companions: Root-Management-v4.20.1, Kernel-Refactor-Analysis-v0.2 (the convergence-plane program), Soak-Framework-Overview-v4.21.0.